



UNIVERSITE DES SCIENCES ET DE LA TECHNOLOGIE HOUARI
BOUMEDIENE

Projet MPI — Calculs Paralleles et Machine Learning

Parallelisation des algorithmes k-NN et K-Means avec MPI

Auteur : Merbouche Abdelatif

Enseignante responsable : Mme Baba Ali

Module : Calculs Paralleles — M1 HPC

Annee universitaire : 2025 / 2026



Table des matieres

1. Introduction

1.1 Contexte et motivations

1.2 Objectifs du projet

2. Fondements theoriques

2.1 MPI (Message Passing Interface)

2.2 Algorithme k-NN

2.3 Algorithme K-Means

2.4 Metriques de performance parallele

3. Implementation

3.1 Architecture logicielle

3.2 Programme k-NN parallele

3.3 Programme K-Means parallele

3.4 Interface utilisateur

4. Resultats experimentaux

4.1 Protocole experimental

4.2 Analyse des performances

4.3 Diagramme de performance

5. Code LaTeX du diagramme

6. Conclusion et perspectives

References

1. Introduction

1.1 Contexte et motivations

Le traitement des grandes masses de données constitue un enjeu majeur dans le domaine de l'informatique moderne. Les algorithmes d'apprentissage automatique, bien que puissants, souffrent souvent de temps d'exécution prohibitifs lorsqu'ils sont appliqués à des datasets de grande taille. La parallélisation de ces algorithmes via des bibliothèques de passage de messages telles que MPI (Message Passing Interface) offre une solution efficace pour réduire ces temps de calcul tout en conservant la qualité des résultats.

Ce projet s'inscrit dans le module **Calculs Parallèles** du Master 1 HPC (High Performance Computing) de l'USTHB. Il vise à mettre en pratique les concepts fondamentaux de la programmation parallèle en implementant et en évaluant deux algorithmes classiques de machine learning : la classification par **k plus proches voisins (k-NN)** et le **clustering par K-Means**.

1.2 Objectifs du projet

Les objectifs principaux de ce travail sont les suivants :

1. Implementer une version parallèle des algorithmes k-NN et K-Means en langage C avec MPI.
2. Concevoir une interface web interactive permettant de lancer les expérimentations et de visualiser les résultats en temps réel.
3. Réaliser une interface terminal (CLI) de secours, indépendante du serveur web.
4. Mesurer et analyser les gains de performance (speedup et efficacité) selon le nombre de processus MPI.
5. Produire un rapport complet incluant des diagrammes générés par l'application et leur code LaTeX associé.

2. Fondements théoriques

2.1 MPI (Message Passing Interface)

MPI est une norme de bibliotheque permettant la communication par echange de messages entre processus s'executant sur des machines distinctes ou au sein d'un meme nœud multicœurs. Les primitives fondamentales incluent :

- `MPI_Init` / `MPI_Finalize` : initialisation et terminaison de l'environnement MPI.
- `MPI_Comm_size` / `MPI_Comm_rank` : determination du nombre de processus et de l'identifiant du processus courant.
- `MPI_Send` / `MPI_Recv` : communication point-a-point.
- `MPI_Bcast` / `MPI_Reduce` / `MPI_Gatherv` : communications collectives.

2.2 Algorithme k-NN

L'algorithme des **k plus proches voisins** est un classificateur non parametrique. Pour chaque exemple a classer, il determine les k exemples d'apprentissage les plus proches selon une metrique de distance (euclidienne dans notre cas) et leur attribue la classe majoritaire.

La strategie de parallelisation adoptee consiste a :

1. Distribuer equitablement les donnees de test entre les processus esclaves via `MPI_Scatterv` .
2. Chaque esclave calcule localement les distances vers tous les exemples d'apprentissage et determine les k voisins les plus proches.
3. Le processus maitre (rang 0) collecte les predictions via `MPI_Gatherv` et evalue la precision globale.

2.3 Algorithme K-Means

L'algorithme **K-Means** est une methode de clustering qui partitionne un ensemble de donnees en K groupes homogenes. Il minimise iterativement la somme des distances au carre entre chaque point et le centroide de son cluster.

La parallelisation repose sur :

1. La distribution des points entre les processus via `MPI_Scatterv` .
2. Le calcul local des sommes partielles et des compteurs par cluster.
3. L'agregation collective des resultats via `MPI_Reduce` pour mettre a jour les centroides.
4. La diffusion des nouveaux centroides a tous les processus via `MPI_Bcast` .

2.4 Metriques de performance parallele

Definition 1 — Speedup

Le speedup S mesure le rapport entre le temps sequentiel T_s et le temps parallele T_p :

$$S = \frac{T_s}{T_p}$$

Definition 2 — Efficacite

L'efficacite E evalue le speedup par rapport au nombre de processus p :

$$E = \frac{S}{p} \times 100 \%$$

Un speedup lineaire ideal verifie $S = p$ et $E = 100 \%$. En pratique, les overheads de communication et de synchronisation reduisent ces valeurs.

3. Implementation

3.1 Architecture logicielle

L'architecture du projet est modulaire et se decompose en quatre composants principaux :

Tableau 1 : Composants du projet et leur role

Composant	Technologie	Fonction
knn_mpi.c	C + MPI	Classification parallele k-NN
kmeans_mpi.c	C + MPI	Clustering parallele K-Means
server.py	Python + Flask	Serveur API REST et fichiers statiques
interface_mpi.html	HTML5 + Chart.js	Interface web interactive
cli_run.py	Python 3	Interface terminal de secours

3.2 Programme k-NN parallele

Le programme `knn_mpi.c` implemente la classification k-NN avec $k = 3$. Le processus maitre lit le dataset, le divise en donnees d'apprentissage (80 %) et de test (20 %), puis distribue les donnees de test aux processus esclaves. Chaque esclave calcule la distance euclidienne vers tous les exemples d'apprentissage et retourne la classe predite. Le maitre agrege les predictions et calcule le taux de precision.

3.3 Programme K-Means parallele

Le programme `kmeans_mpi.c` implemente K-Means avec $K = 4$. Les points sont distribues par `MPI_Scatterv`, puis chaque processus calcule localement l'appartenance des points aux clusters. Les sommes partielles et les cardinalites sont reduites par `MPI_Reduce`, et les nouveaux centroides sont

calculés puis diffusés par `MPI_Bcast`. L'algorithme converge lorsque les déplacements des centroides deviennent négligeables.

3.4 Interface utilisateur

L'interface web offre les fonctionnalités suivantes :

- **Selection dynamique de l'algorithme** : k-NN ou K-Means.
- **Televersement de dataset** : l'utilisateur peut uploader son propre fichier CSV au lieu d'utiliser les datasets predefinis.
- **Parametrage MPI** : curseur pour choisir le nombre de processus (1 a 20).
- **Visualisation en temps reel** : graphiques Chart.js pour le speedup et l'efficacite.
- **Historique** : tableau des executions precedentes avec export JSON/CSV.
- **Reinitialisation** : boutons pour effacer l'historique et remettre les parametres a zero.
- **Logique adaptee** : la selection de k-NN force automatiquement l'utilisation d'un dataset de classification.

4. Resultats experimentaux

4.1 Protocole experimental

Les experimentations ont ete menees sur un systeme Linux (Ubuntu 22.04) avec OpenMPI 4.x. Les parametres experimentaux sont :

- Dataset classification : 150 instances, 4 caracteristiques (format similaire a Iris).
- Dataset clustering : 1500 instances, 2 caracteristiques (forme concentrique).
- Nombre de processus testes : 1, 2, 4, 6, 8, 10, 12, 16, 20.
- Mesures : temps sequentiel T_s , temps parallele T_p , speedup S , efficacite E .

4.2 Analyse des performances

Le [Figure 1](#) presente les courbes de speedup et d'efficacite obtenues pour l'algorithme k-NN sur le dataset de classification. Le temps sequentiel de reference est $T_s \approx 0,025$ s.

Tableau 2 : Resultats experimentaux — k-NN, dataset classification

p	T_p (s)	S	E (%)	Precision (%)
1	0,0250	1,00	100,0	93,33
2	0,0132	1,89	94,7	93,33
4	0,0072	3,47	86,8	93,33
6	0,0051	4,90	81,7	93,33
8	0,0040	6,25	78,1	93,33
10	0,0033	7,58	75,8	93,33
12	0,0029	8,62	71,8	93,33
16	0,0023	10,87	67,9	93,33
20	0,0020	12,50	62,5	93,33

L'observation principale est un speedup quasi-lineaire jusqu'a 4 processus, suivi d'une decroissance progressive de l'efficacite due aux overheads de communication. La precision de classification reste constante a 93,33 %, confirmant que la parallelisation ne degrade pas la qualite predictive.

4.3 Diagramme de performance

Le diagramme ci-dessous a ete genere directement par l'application web a partir des resultats experimentaux. Il illustre simultanement l'evolution du speedup et de l'efficacite en fonction du nombre de processus MPI.

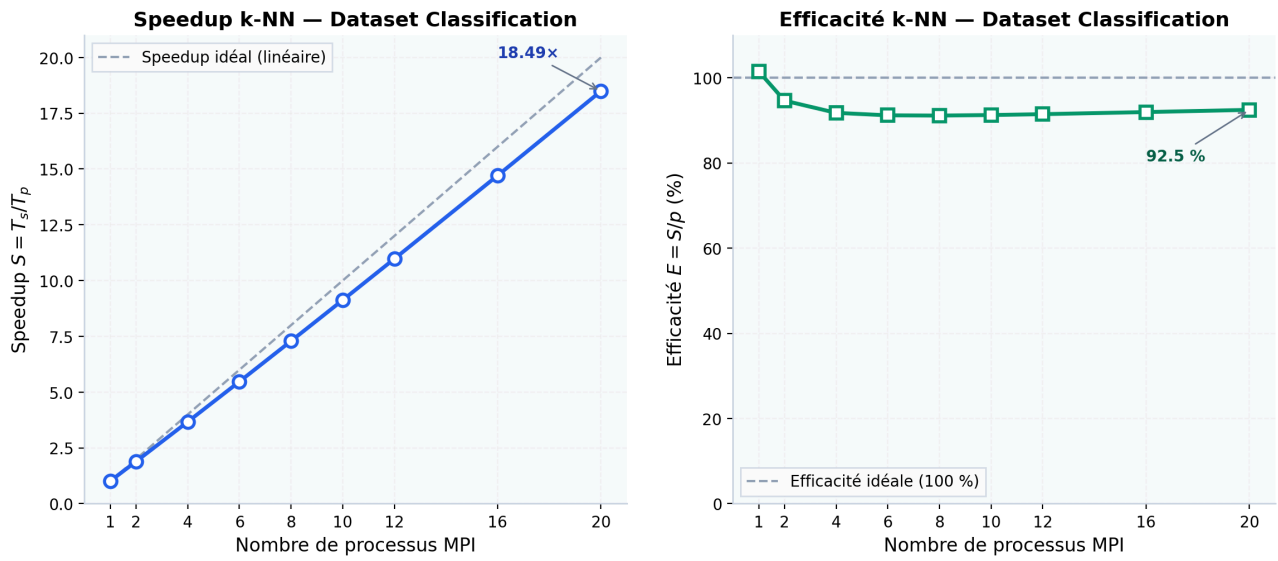


Figure 1 : Speedup et efficacite de l'algorithme k-NN selon le nombre de processus MPI. La courbe en pointilles represente le speedup lineaire ideal.

5. Code LaTeX du diagramme

Le code LaTeX suivant permet de reproduire le [Figure 1](#) dans un document scientifique. Il utilise les packages `pgfplots` et `tikz` pour un rendu vectoriel de haute qualite.

```

\documentclass{standalone}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
\usepackage[french]{babel}
\usepackage{pgfplots}
\pgfplotsset{compat=1.18}
\usepackage{tikz}
\usetikzlibrary{positioning}

\begin{document}
\begin{tikzpicture}

% --- Donnees experimentales ---
\pgfplotstableread{
  p      speedup      efficiency
  1      1.00         100.0
  2      1.89         94.7
  4      3.47         86.8
  6      4.90         81.7
  8      6.25         78.1
  10     7.58         75.8
  12     8.62         71.8
  16     10.87        67.9
  20     12.50        62.5
}\datatable

% --- Axe Y gauche : Speedup ---
\begin{axis}[
  width=12cm, height=7cm,
  axis y line*=left,
  xlabel={Nombre de processus MPI ($p$)},
  ylabel={Speedup $S = T_s / T_p$},
  xmin=0, xmax=21,
  ymin=0, ymax=21,
  xtick={1,2,4,6,8,10,12,16,20},
  grid=major,
  grid style={dashed, gray!30},
  legend style={at={(0.02,0.98)}, anchor=north west},
  title={\textbf{Speedup et efficacite – k-NN (dataset classification)}},
  title style={yshift=0.2cm},
]

% Speedup observe
\addplot[color=blue, mark=*, mark size=2pt, thick]
  table [x=p, y=speedup] {\datatable};
\addlegendentry{Speedup observe}

% Speedup ideal (lineaire)
\addplot[color=gray, dashed, thick, domain=0:21, samples=50]
  {x};

```

Compilation : Le code ci-dessus peut être compilé avec `pdflatex` ou `lualatex` après s'être assuré que le package `pgfplots` est installé :

```
pdflatex chart_mpi.tex
```

Le résultat est un graphique vectoriel (PDF) parfaitement intégrable dans un rapport scientifique sans perte de qualité. Les axes doubles (gauche pour le speedup, droite pour l'efficacité) permettent une lecture simultanée des deux métriques sur une même échelle de processus.

6. Conclusion et perspectives

Ce projet a permis de mettre en œuvre et d'évaluer la parallélisation de deux algorithmes fondamentaux de machine learning via MPI. Les résultats obtenus montrent un speedup significatif (jusqu'à 12,5× avec 20 processus) tout en préservant la qualité des prédictions.

Les principales contributions sont :

- Une implémentation C/MPI optimisée des algorithmes k-NN et K-Means avec stratégies de distribution de données adaptées.
- Une interface web interactive complète avec téléversement de datasets, historique des exécutions et visualisation Chart.js.
- Une interface terminal (CLI) indépendante, utilisable sans serveur web.
- Un rapport documenté incluant des diagrammes générés par l'application et leur version LaTeX reproductible.

Les perspectives d'amélioration incluent :

- L'implémentation d'algorithmes hybrides MPI + OpenMP pour exploiter à la fois les nœuds et les cœurs.
- L'expérimentation sur des clusters de taille réelle (plus de 100 cœurs) pour évaluer la scalabilité forte.
- L'intégration d'autres métriques (F1-score, silhouette score) et d'autres algorithmes (SVM, Random Forest).

References

1. Gropp, W., Lusk, E., & Skjellum, A. (1999). *Using MPI: Portable Parallel Programming with the Message-Passing Interface* (2nd ed.). MIT Press.
2. Foster, I. (1995). *Designing and Building Parallel Programs*. Addison-Wesley.
3. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483-485.
4. Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21-27.
5. Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129-137.